rag -- 2026-10-31

5 new paper(s) · topic: Retrieval-augmented generation: retrieval quality, grounding and faithfulness, evaluation benchmarks, and RAG system design.

Worth reading in full

[P1] Glance: The Generator Is Its Own Reranker -- Glance provides a concrete, model-centric RAG design that unifies generation and reranking, includes detailed latency-memory trade-offs, and reports extensive relevance metrics, making it the most broadly applicable reference for system architects.

Themes

Several papers focus on redesigning the retrieval component to improve relevance while reducing system complexity. Glance shows that a single 9B generator can serve as its own reranker via a lightweight LoRA adapter, achieving nDCG@10 = 0.741 on SciFact and a 0.045 F1 lift on HotpotQA, and it removes the separate reranker backbone though its latency per query remains higher than that of a cross-encoder [P1]. GraphRAG and TCC-RAG both enrich retrieval with structured knowledge: GraphRAG adds a knowledge-graph layer that dramatically outperforms dense-vector RAG on multi-hop and entity-resolution queries in an SEC-EDGAR benchmark, but its gains are limited to that domain [P2]; TCC-RAG builds a causal-cue graph over a temporal KG, delivering 10.7 % and 8.4 % absolute improvements on MULTITQ and TimeQuestions and a 62.9 % boost for multi-step temporal reasoning [P3]. QESChunker treats chunking as a supervised combinatorial optimization problem and attains the highest evidence-coverage scores across five corpora [P5]. Together, these works illustrate a shift from purely similarity-based retrieval toward hybrid designs that combine generator-side scoring, graph-structured metadata, and task-aware chunking to improve grounding and faithfulness, while also exposing evaluation gaps such as limited domain coverage (GraphRAG) and high memory demands for large candidate sets (Glance).

3 theme sentence(s) checked against the paper summaries; 2 corrected, 0 removed.

At a glance

[P1] Glance: The Generator Is Its Own Reranker · relevance 0.41
Glance lets a single 9B generator act as its own reranker using a lightweight adapter, achieving competitive relevance scores while reducing model duplication and latency.

[P2] Beyond Similarity Search: Knowledge Graph Metadata Enrichment for Enterprise Retrieval-Augmented Generation · relevance 0.40
GraphRAG outperforms text-to-SQL and dense-vector RAG on multi-hop and entity-resolution queries in a SEC-EDGAR corpus, but its advantage is limited to this single domain and small benchmark.

[P3] TCC-RAG: Temporal Causal Chain-Guided RAG for Temporal Knowledge Graph Question Answering · relevance 0.40
TCC-RAG introduces a causal-chain guided retrieval framework for temporal knowledge graph question answering that yields large gains, especially on multi-step temporal reasoning.

[P4] Incentivizing Self-Learning in Large Language Models via Reinforcement Learning · relevance 0.39
Reinforcement learning dramatically improves large language models' ability to self-learn from textbooks, raising proof accuracy from 51.5 % to 79.5 % and boosting performance on long-context benchmarks.

[P5] QESChunker: A Single Objective Unifies Overlapping and Non-Overlapping Chunking for RAG · relevance 0.39
QESChunker introduces a supervised, budget-aware chunking method that outperforms existing heuristics on five RAG corpora, using exact dynamic programming for non-overlapping chunks and a submodular greedy algorithm for overlapping chunks.

[P1] Glance: The Generator Is Its Own Reranker

Adilet Metinov, Chingiz Arziev · arXiv 2610.40454v1 · https://arxiv.org/abs/2610.40454v1 · relevance 0.41

TL;DR: Glance lets a single 9B generator act as its own reranker using a lightweight adapter, achieving competitive relevance scores while reducing model duplication and latency.

Problem
Retrieval-augmented generation typically relies on separate models for passage reranking and answer generation, which duplicates model residency and passage processing overhead[1]. Glance removes the additional reranker backbone while delivering competitive retrieval quality [2].

Method
Glance augments a frozen 9B backbone with a rank-16 LoRA decision adapter and a pointer head that scores passage-specific probes attached to a shared prefill of all candidate passages[1][2]. Two pipelines are offered: (i) *Glance (main)* where the adapter is active during probing, then turned off for re-prefilling the top-n passages before answering; and (ii) *Glance-Activated* where the adapter is active only on probe tokens and generation continues from the original cache without re-prefilling[2][3]. The system processes all k candidates in a single "single-pass" prefill, producing k relevance scores without separate query-passage encodings[3]. The backbone interleaves Gated DeltaNet linear-attention layers with full-attention layers to support shared-prefix caching[3].

Key results
- After 39 min of MS MARCO fine-tuning, Glance attains nDCG@10 = 0.741 on SciFact and 0.345 on NFCorpus, surpassing the bge-reranker-v2-m3 baselines (0.721 and 0.326)[1].
- On 300 HotpotQA questions, using Glance-selected context improves answer F1 by +0.045 (95 % CI [0.014, 0.076]) over cross-encoder-selected context under the same generator[1].
- In a scaling study, scoring up to 200 candidates (~43 k tokens) in one shared prefill yields nDCG@10 = 0.717, exceeding the cross-encoder's 0.659 by +0.058[1][4].
- Single-pass reranking is 3.6 × faster than 20 isolated pointwise calls on a 9B model and close to the dedicated cross-encoder (0.721 nDCG@10) on SciFact, while latency per query remains higher than the cross-encoder's 31-60 ms[4].
- Training the adapter on MS MARCO for 39 min (zero-shot) and an additional 14 min on longer contexts (trained-long) improves scores across k values; e.g., at k = 100, trained-long reaches nDCG@10 = 0.697 vs. cross-encoder 0.637, and at k = 200 reaches 0.717 vs. 0.659[4][5].
- Memory usage grows with k, reaching 56 GiB at k = 100[5].
- The activated-adapter variant attains nDCG@10 = 0.740/0.351 on SciFact/NFCorpus (trained) versus 0.744/0.351 for a control adapter, showing comparable reranking quality[6].
- For HotpotQA, continuation from the shared cache yields EM/F1 = 0.61/0.750, slightly lower than re-prefilling top-3 passages (0.59/0.754) but with reduced latency (138 ms vs. 181 ms)[6].

Limitations
- Evaluation is limited to two retrieval datasets (SciFact, NFCorpus), one QA dataset (HotpotQA), and a single 9B backbone family[1][7].
- BEIR differences were not significance-tested; gaps below ~0.02 nDCG@10 are only indicative[4].
- Memory scales super-linearly (e.g., 56 GiB at k = 100), constraining candidate counts and requiring chunked processing for k = 200[5].
- The cross-encoder remains faster per query despite Glance's competitive quality[2][4].
- Scores exhibit order sensitivity in the packed_all mode, with nDCG@10 varying from 0.678 to 0.693 across passage orders[6].
- The paper does not report results on larger benchmarks, other backbone sizes, or beyond 200 candidates, leaving open how benefits extend to broader settings[7].

Sources

[1] arXiv 2610.40454v1.pdf, arXiv abstract

[2] arXiv 2610.40454v1.pdf, page 2

[3] arXiv 2610.40454v1.pdf, page 3

[4] arXiv 2610.40454v1.pdf, page 5

[5] arXiv 2610.40454v1.pdf, page 7

[6] arXiv 2610.40454v1.pdf, page 9

[7] arXiv 2610.40454v1.pdf, page 10

[P2] Beyond Similarity Search: Knowledge Graph Metadata Enrichment for Enterprise Retrieval-Augmented Generation

Fakabbir Amin · arXiv 2610.37728v1 · https://arxiv.org/abs/2610.37728v1 · relevance 0.40

TL;DR: GraphRAG outperforms text-to-SQL and dense-vector RAG on multi-hop and entity-resolution queries in a SEC-EDGAR corpus, but its advantage is limited to this single domain and small benchmark.

Problem
Retrieval-augmented generation (RAG) for enterprise documents struggles when information is spread across multiple filing types, teams, and time periods, making similarity-only search insufficient and leading to ambiguous entity identification [1][2].

Method
The study compares three retrieval architectures--text-to-SQL over typed columns, dense-vector RAG using HNSW cosine similarity, and a two-stage GraphRAG that traverses a knowledge graph of 430,257 nodes and 561,144 edges--under identical model and temperature settings [1][3]. The experimental corpus is a quarterly snapshot of the SEC EDGAR system containing 167,050 filings from 10,511 companies, 508,714 embedded chunks of Item 1A risk-factor text, and the aforementioned graph [1][3]. Evaluation uses 20 manually crafted questions covering four shapes (single-document, structured aggregation, multi-hop relational, cross-document entity resolution), each asked three times for a total of 180 scored attempts; scoring is deterministic, based on entity recall, keyword presence, and citation correctness, without an LLM judge [1][4][5].

Key results
- Overall, GraphRAG solved 16 / 20 questions on all three attempts and passed 50 / 60 attempts, versus 10 / 20 (36 / 60 passes) for text-to-SQL and 3 / 20 (9 / 60 passes) for vector RAG [5][7].
- Category T1 (single-document semantic): GraphRAG 15 / 15, vector RAG 3 / 15, text-to-SQL 7 / 15 [5].
- Category T2 (structured aggregation): text-to-SQL 15 / 15, GraphRAG 6 / 15, vector RAG 0 / 15 [5].
- Category T3 (multi-hop relational + text): GraphRAG 14 / 15, both baselines 0 / 15 [5].
- Category T4 (cross-document entity resolution): GraphRAG 15 / 15, text-to-SQL 14 / 15, vector RAG 6 / 15 [5].
- GraphRAG produced no confident falsehoods, whereas text-to-SQL generated six [1][5].
- GraphRAG always provided provenance citations; the baselines were uncited 10 and 12 times respectively across 60 attempts [7].
- The graph-based approach incurred higher token cost for aggregation queries (~12.2 × tokens) compared with text-to-SQL [7].

Limitations
The evaluation is confined to a single SEC-EDGAR snapshot, a domain with mandatory structure and stable CIK identifiers, so results may not transfer to corpora lacking authoritative identifiers [6][7]. The benchmark comprises only 20 questions (five per category), giving wide confidence intervals and limiting statistical strength [6]. Questions were authored by the graph's creator, introducing potential bias despite pre-registration of two losing categories [6]. Consequently, the findings are best viewed as directional insights about question shape rather than a general claim of superiority [1][6][7].

Sources

[1] arXiv 2610.37728v1.pdf, arXiv abstract

[2] arXiv 2610.37728v1.pdf, page 3

[3] arXiv 2610.37728v1.pdf, page 5

[4] arXiv 2610.37728v1.pdf, page 7

[5] arXiv 2610.37728v1.pdf, page 9

[6] arXiv 2610.37728v1.pdf, page 12

[7] arXiv 2610.37728v1.pdf, page 14

[P3] TCC-RAG: Temporal Causal Chain-Guided RAG for Temporal Knowledge Graph Question Answering

Xinxun Zhang, Yuxiang Wang, Shanshan Feng, Jun Zhang et al. · arXiv 2610.38366v1 · https://arxiv.org/abs/2610.38366v1 · relevance 0.40

TL;DR: TCC-RAG introduces a causal-chain guided retrieval framework for temporal knowledge graph question answering that yields large gains, especially on multi-step temporal reasoning.

Problem - Existing semantic-based retrieval-augmented generation (RAG) methods for TKGQA fail to capture the multi-hop causal knowledge needed for complex temporal reasoning and often retrieve excessive redundant facts that can trap large language models in spurious reasoning chains [1]. Moreover, GraphRAG approaches have not been thoroughly explored for supporting multi-step temporal reasoning, and current embedding-based and LLM-based TKGQA methods still rely on semantic similarity retrieval, which misses crucial causal links [2].

Method - TCC-RAG first reformulates the temporal knowledge graph into a multi-view causal cue graph that explicitly encodes temporal evolution and causal dynamics [1]. For each query, a query-specific temporal causal chain is constructed by routing the query to the appropriate view (Temporal Event Graph, Subject Causal Cue Index, or Object Causal Cue Index) and retrieving cue-consistent events; a causal evidence filter then refines the retrieved set to ensure causal consistency with the query's temporal logic [4]. The approach also defines a routing function ? that selects the view based on the answer slot and grounded entities [4].

Key results - On the two widely used TKGQA benchmarks, MULTITQ (over 500 K questions) and TimeQuestions (~16 K questions), TCC-RAG achieves performance gains of 10.72 % and 8.40 % respectively [1][5][6]. For multi-step temporal reasoning, it delivers a 62.97 % improvement over prior methods [1]. The experiments compare against a diverse set of baselines, including pretrained language models (e.g., BERT, DistilBERT, ALBERT) and other TKGQA approaches [6].

Limitations - The authors note that ambiguous entity mentions and misaligned relations can lead to empty or incorrect structured evidence, and while semantic retrieval offers a fallback, it does not guarantee detection of erroneous evidence [5]. Additionally, on sparse knowledge graphs the SCCI/OCCI indices provide limited benefit because posting lists often contain a single entry, causing multi-step causal chain retrieval to degenerate to single-fact lookup; adapting TCC-RAG to such settings is left for future work [5].

Sources

[1] arXiv 2610.38366v1.pdf, arXiv abstract

[2] arXiv 2610.38366v1.pdf, page 3

[4] arXiv 2610.38366v1.pdf, page 6

[5] arXiv 2610.38366v1.pdf, page 8

[6] arXiv 2610.38366v1.pdf, page 8

[P4] Incentivizing Self-Learning in Large Language Models via Reinforcement Learning

Lingkai Kong, Yuzhe Gu, Haiteng Zhao, Zijian Wu et al. · arXiv 2610.39671v1 · https://arxiv.org/abs/2610.39671v1 · relevance 0.39

TL;DR: Reinforcement learning dramatically improves large language models' ability to self-learn from textbooks, raising proof accuracy from 51.5 % to 79.5 % and boosting performance on long-context benchmarks.

Problem
Current large language models rely on static parametric knowledge or simple retrieval, which is insufficient for solving unfamiliar, knowledge-intensive tasks that require active acquisition and reasoning over long-form materials [1]. The authors therefore define a textbook-based mathematical proof-solving task in which models must study, learn, and apply principles from structured reference textbooks without seeing the reference solutions [2].

Method
The study introduces the BookNavigator interface that lets the model perform three textbook-access actions--getting the table of contents, keyword search, and reading full sections--to mimic human study behavior [2]. A reinforcement-learning (RL) framework supplies fine-grained rewards for both final proof correctness and intermediate tool usage (tool credit and an exploration bonus) to incentivize self-learning [1][5]. Proof correctness is judged by an LLM-as-judge that compares the model's proof to a hidden reference solution [2].

Key results
- Before RL, the self-learning protocol achieves 51.5 % proof accuracy, below the "No-Book" baseline (55.7 %) and fixed-retrieval baselines [3][4].
- After RL, overall proof accuracy rises to 79.5 %, a 28.0-point improvement, surpassing all baselines [3][4].
- On the BrowseComp-Plus and CL-bench long-context benchmarks, the RL-enhanced model improves base performance by 1.7 and 2.3 percentage points, respectively [1].
- Ablation (Table 4) shows:
* Base policy: 33.9 % proof accuracy, 65.2 % tool-use on in-domain books, 1.5 tool calls/trajectory [5].
* Correctness-only RL: 60.7 % accuracy, tool-use drops to 6.7 % and 0.3 calls/trajectory [5].
* Adding tool credit: accuracy 64.0 % (+3.3 pts), tool-use 7.0 % [5].
* Adding exploration bonus: accuracy 61.8 % (+1.1 pts), tool-use 79.9 % [5].
* Full method (tool credit + exploration): 65.5 % overall accuracy (+4.8 pts), 85.2 % accuracy on out-of-domain books, 86.7 % on out-of-domain subjects, with 3.0 tool-using trajectories and 3.5 calls/trajectory [5].

Limitations
The evaluation focuses solely on proof accuracy using an LLM-as-judge, without measuring other aspects such as reasoning quality or human interpretability [2][5]. Experiments are limited to mathematical textbook tasks and two long-context benchmarks (BrowseComp-Plus, CL-bench), so generalization to other domains remains untested [1]. Additionally, correctness-only RL markedly reduces tool-use, suggesting the policy may over-rely on internal knowledge and limit transfer to problems that truly require external study [5].

Sources

[1] arXiv 2610.39671v1.pdf, arXiv abstract

[2] arXiv 2610.39671v1.pdf, page 3

[3] arXiv 2610.39671v1.pdf, page 5

[4] arXiv 2610.39671v1.pdf, page 5

[5] arXiv 2610.39671v1.pdf, page 8

[P5] QESChunker: A Single Objective Unifies Overlapping and Non-Overlapping Chunking for RAG

Yifan Zhao, Qianyang Zhao, Xinyu Guan, kai wei et al. · arXiv 2610.36557v1 · https://arxiv.org/abs/2610.36557v1 · relevance 0.39

TL;DR: QESChunker introduces a supervised, budget-aware chunking method that outperforms existing heuristics on five RAG corpora, using exact dynamic programming for non-overlapping chunks and a submodular greedy algorithm for overlapping chunks.

Problem
Chunking is a crucial preprocessing step for retrieval-augmented generation, yet most existing chunkers rely on heuristic rules rather than task-specific supervision [1]. The authors aim to construct static chunk indices that maximize evidence recall for question-answering without access to held-out test queries [1].

Method
QESChunker formulates chunking as a supervised combinatorial optimization problem using a unified question-evidence utility objective [1]. For non-overlapping partitions the objective decomposes and can be solved exactly in polynomial time by dynamic programming [2]; for overlapping indices the objective is monotone submodular, so the authors apply a benefit-cost greedy algorithm with a (1 - 1/?e) ~ 0.393 approximation guarantee [4]. The greedy method includes a singleton safeguard and lazy evaluation to keep computation practical [4].

Key results
- Across two self-constructed Chinese policy corpora and three public English corpora, the non-overlap DP variant ranks first in 9 / 10 corpus × tier cells, while the overlap greedy variant ranks first in all 10 / 10 cells [2].
- In the non-overlap setting, QESChunker (exact DP) achieves complete-evidence coverage of 0.9025 @ 100 atoms and 0.9781 @ 10 % on the Congress report, 0.8216 @ 100 and 0.9285 @ 10 % on the White paper, and comparable gains on QASPER, PeerQA, and PrivacyQA (see Table 3) [5].
- The largest absolute gains over the best baseline occur on the White paper (+3.98 points at @ 100, +2.61 points at @ 10 %) [5].
- In the overlap setting, QESChunker (greedy) attains coverage of 0.9165 @ 100 and 0.9781 @ 10 % on the Congress report, 0.8656 @ 100 and 0.9545 @ 10 % on the White paper, and similarly strong results on the other corpora (see Table 4) [5].
- The greedy algorithm's empirical gap to the optimum is reported to be much smaller than the theoretical 0.393 bound [4].

Limitations
Exact optimization of the coverage objective is NP-hard, so the non-overlap DP relies on the special structure of non-overlapping partitions, while the overlap case must settle for an approximation guarantee rather than optimality [3][4]. The reported approximation factor (~0.393) is weaker than the best known 0.405 bound for the same algorithm, which the authors deliberately retain [4]. Experiments are conducted under a fixed hybrid retriever and do not include systems that modify retrieval or embedding beyond chunk boundaries, limiting the scope of comparison [2]. Results are reported on five corpora only, so generalization to other domains remains untested.

Sources

[1] arXiv 2610.36557v1.pdf, arXiv abstract

[2] arXiv 2610.36557v1.pdf, page 2

[3] arXiv 2610.36557v1.pdf, page 5

[4] arXiv 2610.36557v1.pdf, page 6

[5] arXiv 2610.36557v1.pdf, page 8